

Computer Architecture Lab 1 Report

CSIE B12902075 沈柏宏

Module Explanations

Control

It reads opcode(instr[6:0]) as input, and outputs three control signals: RegWrite, ALUSrc, ALUOp. This module analyzes the 7-bit opcode, determine which type of instruction the input is, and assigns the corresponding values to signals. My design can only recognize R-type and I-type currently. Also, I set the default output values of (RegWrite, ALUSrc, ALUOp) to be (0,0,2'b00)

Adder

It reads two 32-bit inputs, and simply outputs their sum. When used for PC update, one input is assigned to 32'd4. I did this in combinational logic.

MUX32

It reads two 32-bit inputs, one 1-bit control signal, and output another 32-bit. I simply use an if-else in procedural block to implement the simple logic. Assign the input_0 to output when control is 0, and assign input_1 when it's 1.

Sign_Extend

It reads a 12-bit immediate input (from instr[31:20]), and output extended 32-bit immediate. I used vector arithmetic to yield extended output, just repeat 11-th bit 20 times and concatenate the original 12-bit. I did this in combinational logic.

ALU_Control

It reads 3 inputs: 7-bit funct7 from instr[31:25], 3-bit funct3 from instr[14:12], 2-bit ALUOp from Control module. It outputs the corresponding 3-bit ALU control signal.

From ALUOp, we know the type of this instruction. Knowing instruction type, we can use case to map funct7, funct3 to ALU real operation type according to the table given in spec (or RISC-V manual). For readability, I also use localparam to build a map between operation names and their value to pass (like enum in c language).

ALU

It reads 3 inputs: 2 32-bit operands (one from register, one from MUX (register or immediate)), and 3-bit operation type from ALU Control. It output from 2 internal register: ALU result and zero

The operation name-value table used for encoding at ALU control is also built here for decoding. The procedural logic does the corresponding arithmetic operation based on the operation type received from ALU Control. The arithmetic result is assigned to ALU result register, and zero register is also assigned the boolean whether ALU result is zero.

PC

It reads clock, reset, and next cycle PC as input, and outputs the PC of current cycle. This module assign value from its input PC to output at positive edge of clock. When reset signal is set, output PC value is set to 0.

Instruction_Memory

It reads the 32-bit address of the instruction to fetch, and output the context of the instruction fetched. We declare a instruction memory here using 256 32-bit reg, which will be assigned when running benchmark. To fetch, we use $\text{addr}/4$ as the index because one instruction is 4 byte.

Register

A huge module have several inputs and outputs, but its functionality can be summarized in short:

1. read and output register value given its address
2. modify register value at clock edge if RegWrite signal is given.
3. reset all register value to 0 if reset signal is given.

Synthesizability / latch check screenshot

```
• douglas@local:~/ca_lab1> python3 parse.py
=== Module Areas ===
ALU: 4338.992
ALU_Control: 35.112
Adder: 182.742
Control: 6.916
MUX32: 59.584
PC: 170.240
Registers: 9447.256
=====
Total area: 14240.842

=== Latch Statistics (10.6 PROC_DLATCH) ===
No latch inferred count : 0
Latch inferred count    : 0
```

Development environment

- OS: WSL2, Ubuntu 22.04
- Compiler: Yosys 0.9 (git sha1 1979e0b)
- IDE: Visual Studio Code 1.105.1